
Lab Experiment 7

CI/CD Pipeline using Jenkins, GitHub & Docker Hub

Aim: Design and implement a complete CI/CD pipeline using Jenkins, integrating source code from GitHub, and building & pushing Docker images to Docker Hub.

GitHub repository link

Theor y

What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Deployment/Delivery. It is a modern software development practice that automates the process of building, testing, and deploying code every time a developer makes a change.

Continuous Integration (CI)

Before CI existed, developers would write code separately for days or weeks, then try to merge everything together — this often caused conflicts and bugs that were very hard to find. CI solves this by automatically building and testing the code every time a developer pushes changes to the shared repository.

- Every git push triggers an automated build
- Problems are caught immediately, not weeks later
- The codebase is always in a working, stable state

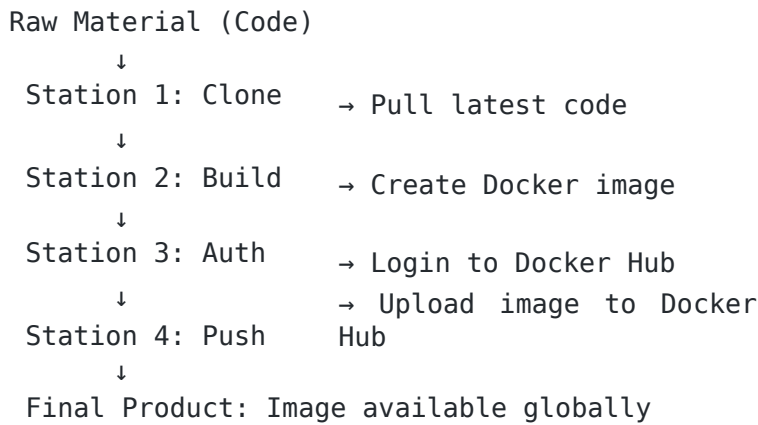
Continuous Deployment (CD)

CD takes the output of CI (a tested, built artifact) and automatically delivers it to its destination — in our case, Docker Hub.

- No manual steps needed after code is pushed
- The Docker image is always up to date on Docker Hub
- Speeds up the release cycle significantly

The CI/CD Pipeline Concept

A pipeline is a sequence of automated steps that code passes through from commit to deployment. Think of it like an assembly line in a factory:



Without CI/CD: Developer pushes code → manually builds image → manually logs into Docker Hub → manually pushes image (slow, error-prone, skippable)

With CI/CD: Developer pushes code → everything else is automatic (fast, consistent, reliable)

What is Jenkins?

Jenkins is an open-source automation server with a browser-based graphical interface (GUI). It acts as the brain of the CI/CD pipeline — it listens for triggers, runs pipeline stages in order, and reports results.

Key features of Jenkins:

- Dashboard UI — View all jobs, their status (success/fail), and build history from a browser

Plugin Ecosystem — Hundreds of plugins to integrate with GitHub, Docker, Slack, AWS, and more

- Pipeline as Code — Pipelines are defined in a Jenkinsfile stored alongside the source code
- Credentials Store — Securely stores passwords and tokens so they are never hardcoded in pipeline scripts
- Agent System — Jenkins can delegate work to other machines (agents); in our experiment, the same Docker host is used as the agent

How Jenkins fits in this experiment:

Jenkins is the orchestrator. It receives the webhook signal from GitHub, reads the Jenkinsfile from the repository, and executes each stage (Clone → Build → Login → Push) one by one automatically.

What is a Webhook?

A webhook is an HTTP-based notification mechanism. Instead of Jenkins constantly checking

("polling") GitHub every few minutes to see if there is new code, GitHub proactively notifies Jenkins the moment a push event happens.

How a webhook works step by step:

1. Developer pushes code to GitHub
2. GitHub immediately sends an HTTP POST request to Jenkins at:
`http://<jenkinsserver>:8080/github-webhook/`
3. Jenkins receives the notification and starts the pipeline automatically

This makes the pipeline event-driven — Jenkins starts building within seconds of a push with zero manual intervention.

Webhook vs Polling:

Method	How it works	Efficiency
Polling	Jenkins checks GitHub every N minutes	Wasteful — checks even when nothing changed
Webhook	GitHub notifies Jenkins instantly needed on push	Efficient — triggers only when

What is a Jenkinsfile?

A Jenkinsfile is a text file that contains the pipeline definition written in Groovy DSL (Domain Specific Language). It tells Jenkins exactly what to do and in what order.

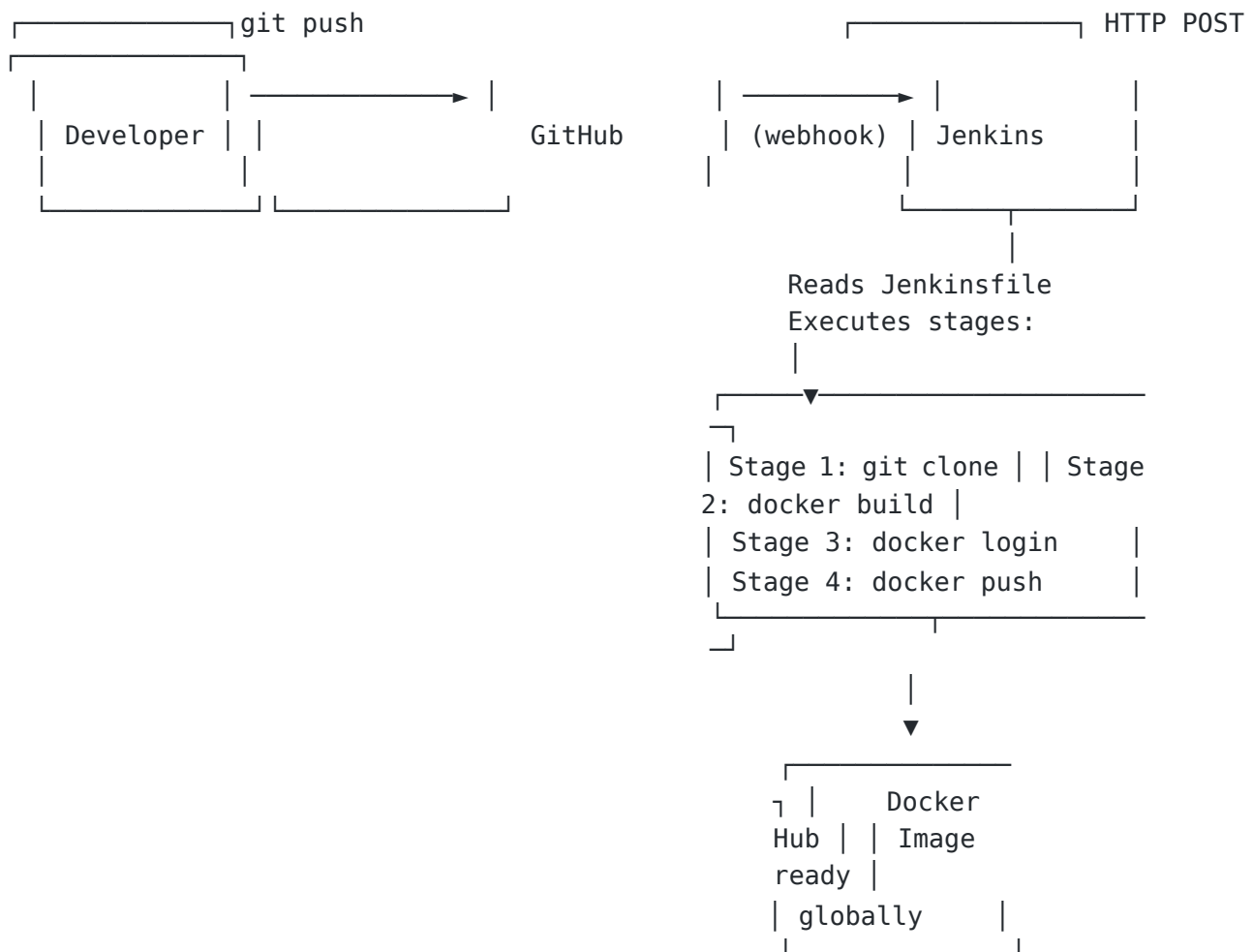
Storing the Jenkinsfile in the repository (alongside the code) means:

- The pipeline is version-controlled — you can see the history of pipeline changes
- Any team member can read and understand how the pipeline works
- If the pipeline breaks after a change, you can roll it back like any other code

The Jenkinsfile follows a declarative syntax with a clear hierarchy:

```
pipeline
├─ agent          ← where to run
├─ environment    ← shared variables
├─ stages
│   └─ stage('Name')
│       └─ steps
│           └─ sh / git / withCredentials / echo
```

How They All Work Together



Prerequisites

- Docker & Docker Compose installed on host machine
- GitHub account with a repository
- Docker Hub account with an access token
- Basic Linux command knowledge

Project Structure

```
my-app/
├── app.py           #Flask web application
├── requirements.txt #Python dependencies
├── Dockerfile       #Docker image build
└── Jenkinsfile      instructions
                    #CI/CD pipeline definition
```

Part A — GitHub Repository Setup

app.py — Detailed Code Explanation

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from CI/CD Pipeline - Updated! from Jenkinsfile"

app.run(host="0.0.0.0", port=80)
```

Code: **app.run(host="0.0.0.0", port=80)** Starts the Flask development web server with two important parameters:

host="0.0.0.0" — Makes the server listen on all available network interfaces, not just the loopback address (127.0.0.1). This is essential inside Docker — without it, the Flask server would only be reachable from inside the container itself and not from the outside world.

port=80 — Runs on port 80, which is the standard HTTP port. This matches the EXPOSE 80 instruction in the Dockerfile.

requirements.txt

```
flask
```

Lists all Python packages needed for the app. When Docker builds the image, it runs `pip install`

`-r requirements.txt`, which reads this file and installs Flask and all its sub-dependencies automatically.

Dockerfile — Detailed Code Explanation

```
FROM python:3.10-slim WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 80
CMD ["python", "app.py"]
```

This Dockerfile creates a lightweight environment to run your Flask app. It starts with a slim

Python 3.10 base image to keep the file size small and sets /app as the active workspace. The COPY command pulls your local code into the container, while RUN installs the necessary dependencies during the build process. Finally, it documents Port 80 as the entry point and uses CMD to automatically launch the script whenever the container is started.

How Docker builds the image layer by layer:

Layer 5: EXPOSE metadata	← EXPOSE 80
Layer 4: flask + deps installed	← RUN pip install
Layer 3: app files copied to /app	← COPY . .
Layer 2: /app directory created	← WORKDIR /app
Layer 1: python:3.10-slim base	← FROM python:3.10-slim
[Runtime – not a layer]: python app.py ← CMD	

```
pipeline{
  agent any

  environment{
    IMAGE_NAME= "anketkiran/myapp"
  }

  stages{
    stage('CloneSource') {
      steps {
        git branch: 'main', url: 'https://github.com/Pavilioni5/my-app.git'
      }
    }

    stage('BuildDocker Image') {
      steps {
        sh 'docker build -t $IMAGE_NAME:latest .'
      }
    }
  }
}
```

Jenkinsfile — Detailed Code Explanation

```
stage('Login to Docker Hub') {
    steps {
        withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) {
            sh 'echo $DOCKER_TOKEN | docker login -u anketkiran --password-stdin'
        }
    }
}

stage('Push to Docker Hub') {
    steps {
        sh 'docker push $IMAGE_NAME:latest'
    }
}
}
```

`pipeline {}` is the main block that defines the entire Jenkins pipeline.

agent any

Specifies where the pipeline runs. `any` means “use whatever build node is available.” In our setup, Jenkins runs inside Docker with the host’s Docker socket mounted, so the pipeline executes directly on the host machine. This means `docker build` and `docker push` commands operate on the real Docker engine of the host.

environment { IMAGE_NAME = "anketkiran/myapp" }

Defines environment variables that are available to all stages throughout the pipeline. Setting

`IMAGE_NAME` here avoids repeating the full image name in every stage. If you need to rename the image, you change it once here.

- Format follows Docker Hub convention: `<dockerhub-username>/<repository-name>`
- Referenced in shell commands as `$IMAGE_NAME`



Stage 1 — 'Clone Source'

```
git branch: 'main', url: 'https://github.com/Pavilion15/my-app.git'
```

The `git` step is a built-in Jenkins step that clones a Git repository into the Jenkins workspace directory. Parameters explained:

`branch: 'main'` — Specifically checks out the main branch (not any other branch like `dev`)

or feature/xyz) url: '...' — The HTTPS URL of the GitHub repository to clone from After this step, all project files (app.py, Dockerfile, requirements.txt, Jenkinsfile) are available in Jenkins' workspace and can be used by subsequent stages.

Stage 2 — 'Build Docker Image'

```
sh 'docker build -t $IMAGE_NAME:latest .'
```

sh runs a shell command on the build agent. Breaking down the full Docker command:

docker build — Instructs Docker to build an image using a Dockerfile
-t \$IMAGE_NAME:latest — Tags the built image. -t stands for tag. \$IMAGE_NAME expands to anketkiran/myapp, making the full tag anketkiran/myapp:latest
. — The build context — tells Docker to look for the Dockerfile in the current directory (the Jenkins workspace where the code was cloned in Stage 1)

Stage 3 — 'Login to Docker Hub'

```
withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN')]) { sh 'echo $DOCKER_TOKEN | docker login -u anketkiran --password-stdin' }
```

This is the most security-critical stage.

withCredentials([...]) A Jenkins DSL function that temporarily retrieves a secret from Jenkins'

encrypted credentials store and makes it available as an environment variable only inside the { } block. Once the block finishes, the variable is destroyed and scrubbed from memory. Jenkins also masks the secret value in build logs, replacing it with ****.

string(credentialsId: 'dockerhub-token', variable: 'DOCKER_TOKEN') This is the credential binding inside the list [...]:

string — Declares the credential type as a secret text string (as opposed to usernamePassword, sshUserPrivateKey, file, etc.)
credentialsId: 'dockerhub-token' — The exact ID used when the token was saved in Jenkins credentials. Jenkins looks up and decrypts the value using this ID.
variable: 'DOCKER_TOKEN' — The name of the temporary environment variable that will hold the decrypted token inside the block


```
echo $DOCKER_TOKEN | docker login -u anketkiran --password-
```

stdin This is a Unix pipe command — two commands connected by | :

Part	What it does
echo \$DOCKER_TOKEN	Prints the token value to standard output
\	Pipes stdout of the left command into stdin of the right command
docker login -u anketkiran	Initiates Docker Hub login for anketkiran
-password-stdin	Reads the password from standard input (the pipe) instead of a command

Stage 4 — 'Push to Docker Hub'

```
sh 'docker push $IMAGE_NAME:latest'
```

Uploads the locally built image to Docker Hub. This works because:

- 1.The previous stage successfully authenticated with Docker Hub (the auth session is stored by Docker on the host)
- 2.The image is tagged as anketkiran/myapp:latest — which maps to the myapp repository under the anketkiran Docker Hub account

After this stage completes, the image is publicly available and can be pulled by anyone with:

```
docker pull anketkiran/myapp:latest
```

Part B — Jenkins Setup using Docker

docker-compose.yml — Detailed Code Explanation

```
version: '3.8'
services:
jenkins:
  image:
jenkins/jenkins:lts
```

```
container_name: jenkins
restart: always
ports:
- "8080:8080"      - "50000:50000"  volumes:

- jenkins_home:/var/jenkins_home
- /var/run/docker.sock:/var/run/docker.sock
user: root
```

```
volumes:
jenkins_home:
```

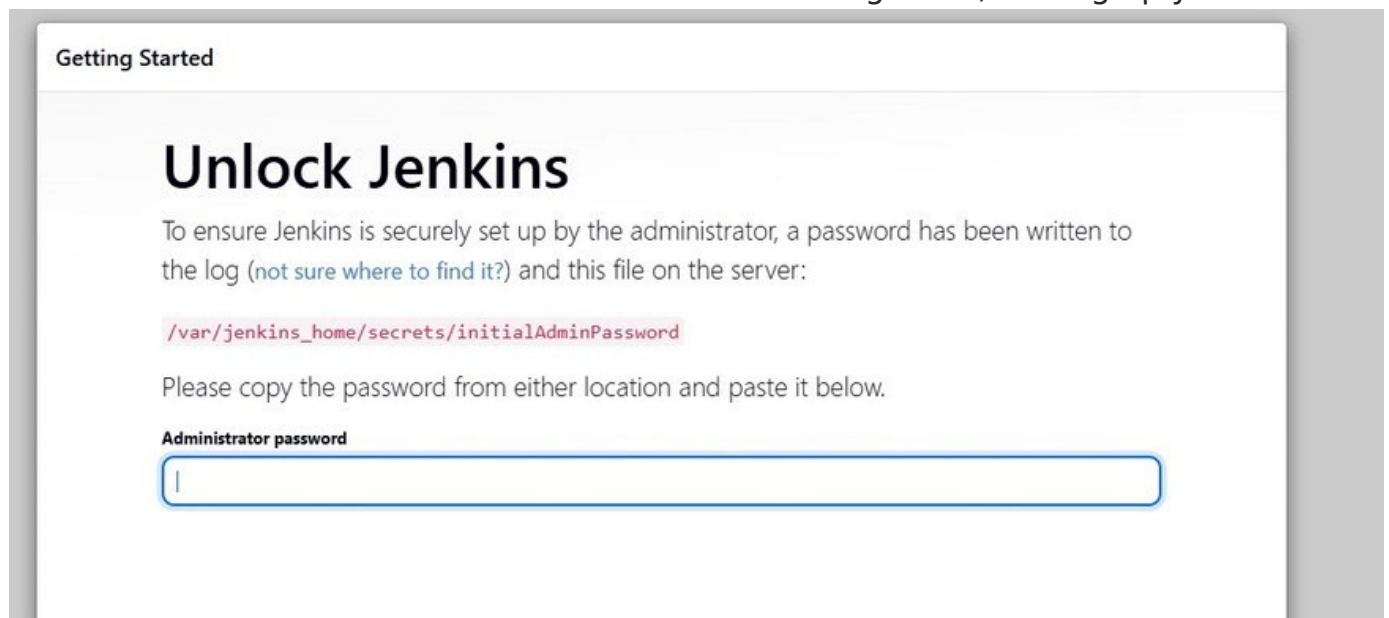
This Docker Compose file configures a stable Jenkins LTS container designed for persistent CI/CD workflows. It ensures high availability with a restart: always policy and maps essential ports for web access (8080) and agent connections (50000). To prevent data loss, a named volume persists all Jenkins configurations and build history on the host machine. Most importantly, it mounts the Docker socket and runs as root, granting Jenkins the necessary permissions to build and manage Docker images directly on the host's engine.

Start Jenkins

```
docker-compose up -d
```

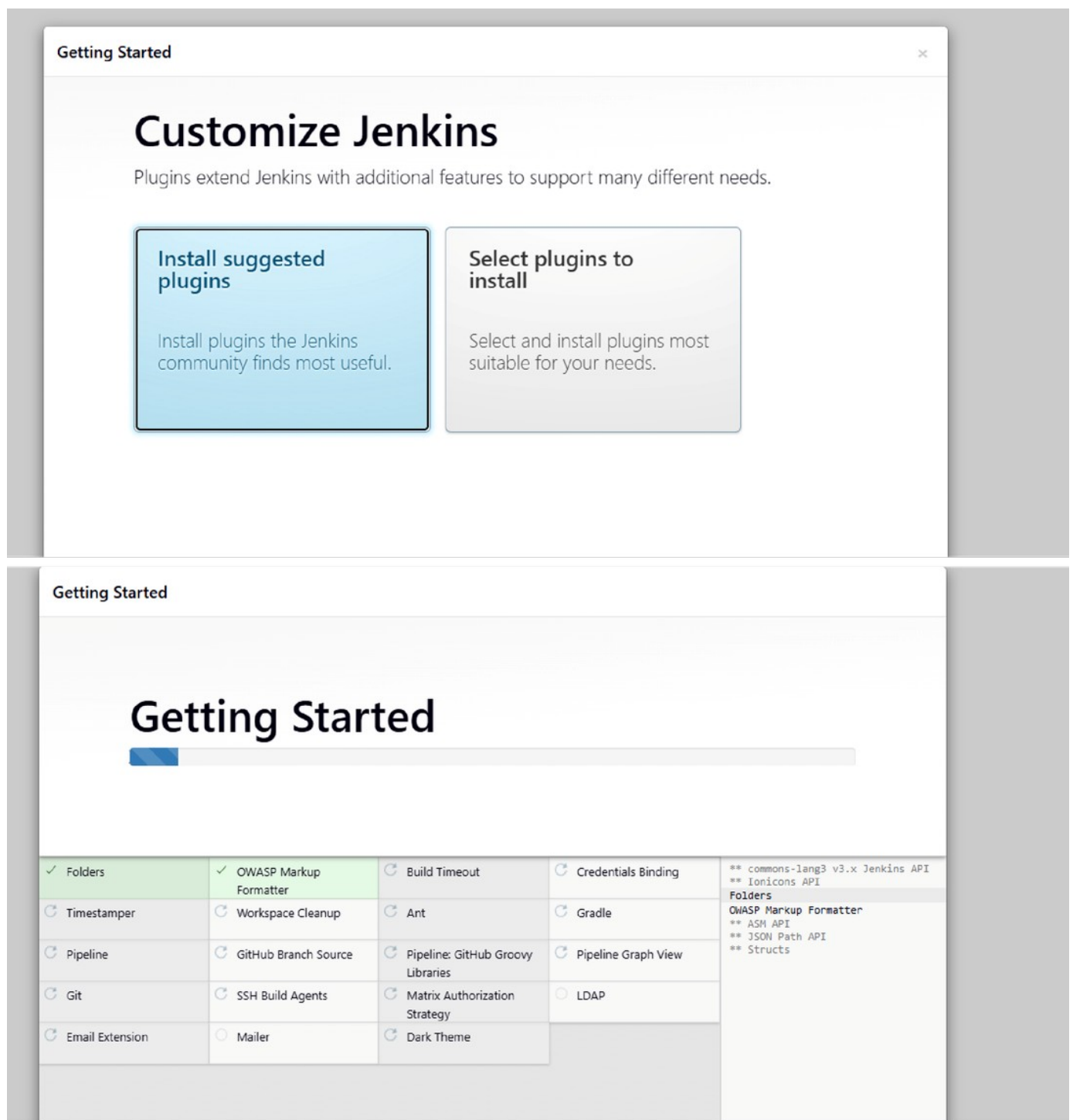
up — Create and start all services defined in docker-compose.yml

-d — Detached mode: run containers in the background, freeing up your terminal



Access Jenkins

`http://localhost:8080`



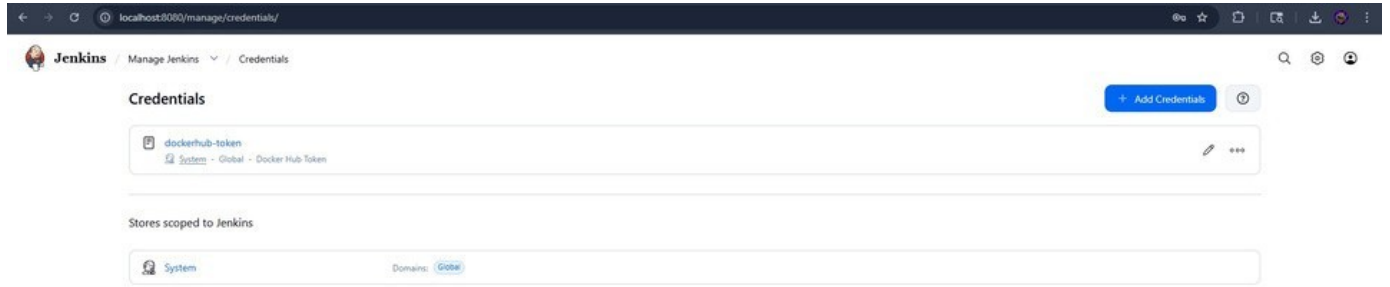
Unlock Jenkins

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

`docker exec` — Execute a command inside a running container

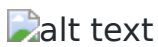
`-it` — Allocate an interactive pseudo-TTY `jenkins` — The container name to target `cat /var/jenkins_home/secrets/initialAdminPassword` — Print the one-time admin

password generated on first startup



Initial Setup Steps

1. Paste the password into the unlock screen
2. Click “Install suggested plugins” and wait for installation to complete
3. Create your admin user (set username and password)
4. Save and proceed to the Jenkins dashboard



Part C — Jenkins Configuration

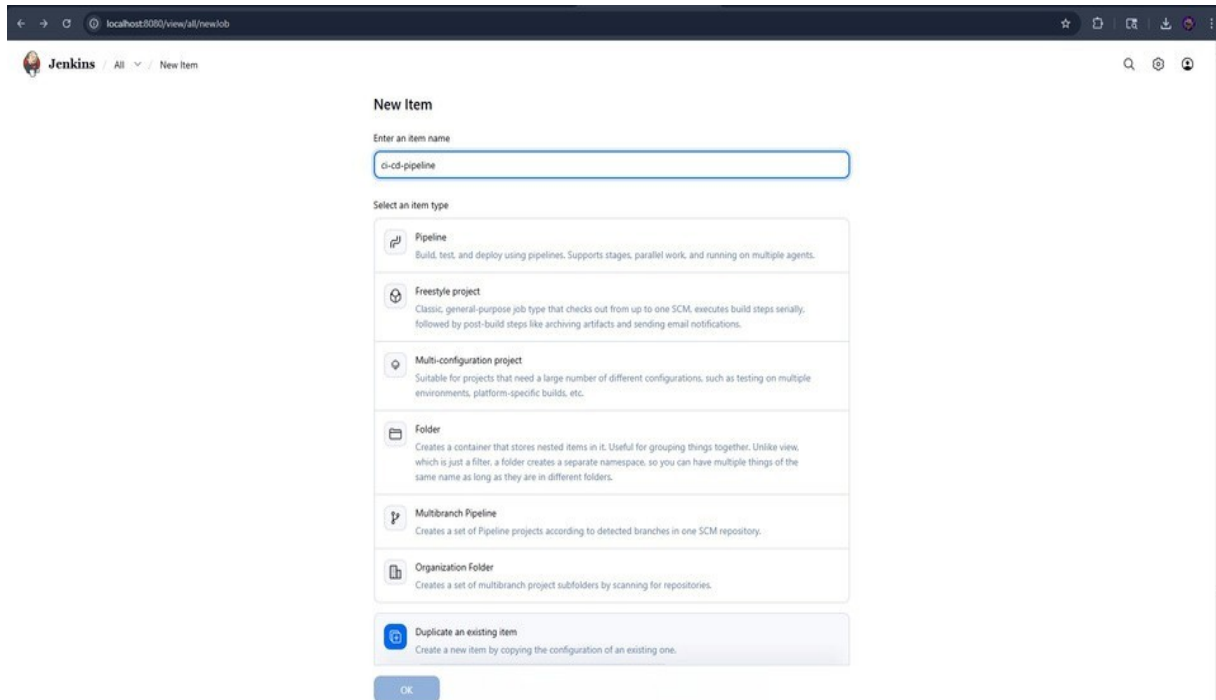
Step 1: Add Docker Hub Credentials

Navigate to:

Manage Jenkins → Credentials → (global) → Add Credentials

Field		Value
Kind	Secret text	
Secret	Your Docker Hub Access Token (from Docker Hub → Account Settings → Security)	
ID	dockerhub-token	
Field		Value

Description Docker Hub Token



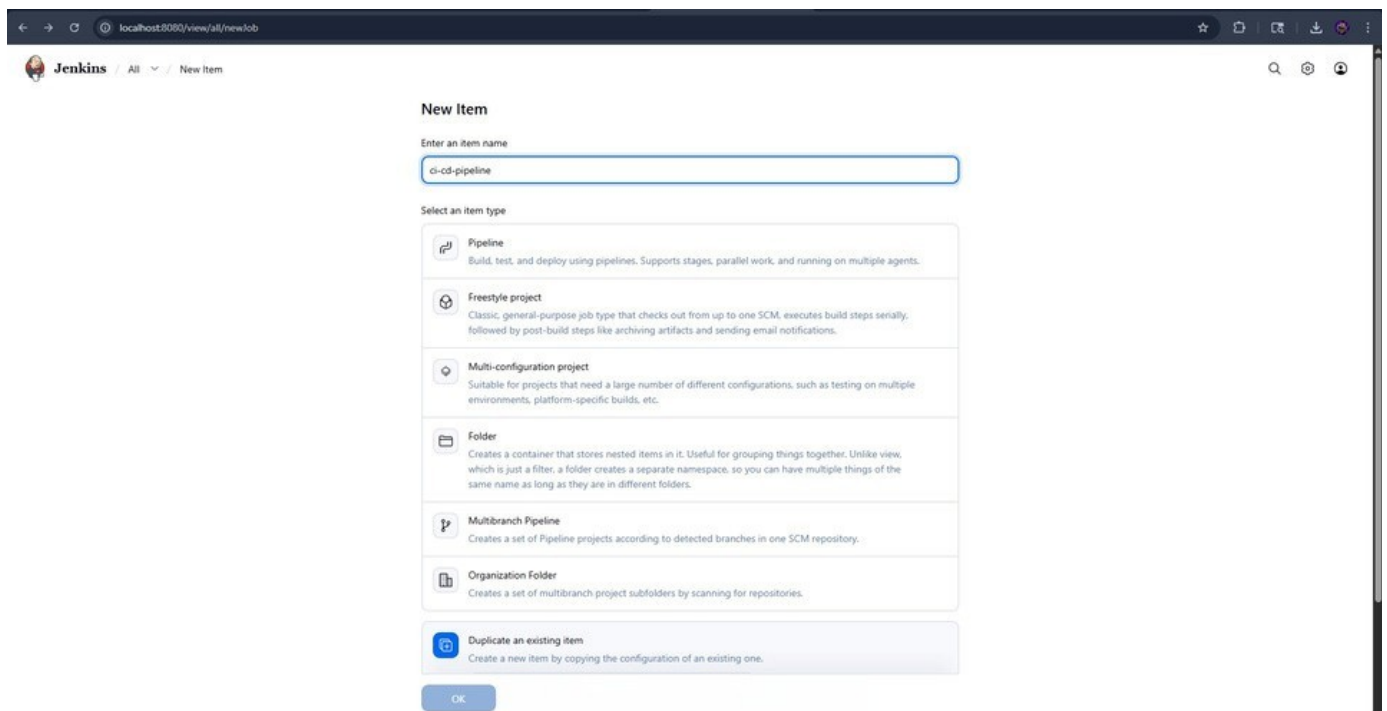
🚩 The `credentialsId` `dockerhub-token` in the Jenkinsfile must exactly match the ID you enter here — including case sensitivity.

Step 2: Create a Pipeline Job

1. Click New Item on the Jenkins dashboard
2. Enter name: ci-cd-pipeline
3. Select Pipeline and click OK
4. Under the Pipeline section, configure:

Field	Value
Definition	Pipeline script from SCM
SCM	Git
Repository URL	<code>https://github.com/Pavilion15/my-app.git</code>
Branch Specifier	<code>*/main</code>
Script Path	Jenkinsfile
1. Click Save	

Why “Pipeline script from SCM”? This tells Jenkins to fetch the Jenkinsfile directly from the GitHub repository rather than copying it into the Jenkins UI. The pipeline definition lives with the code and is version-controlled — changes to the Jenkinsfile are tracked in Git history just like any other file.



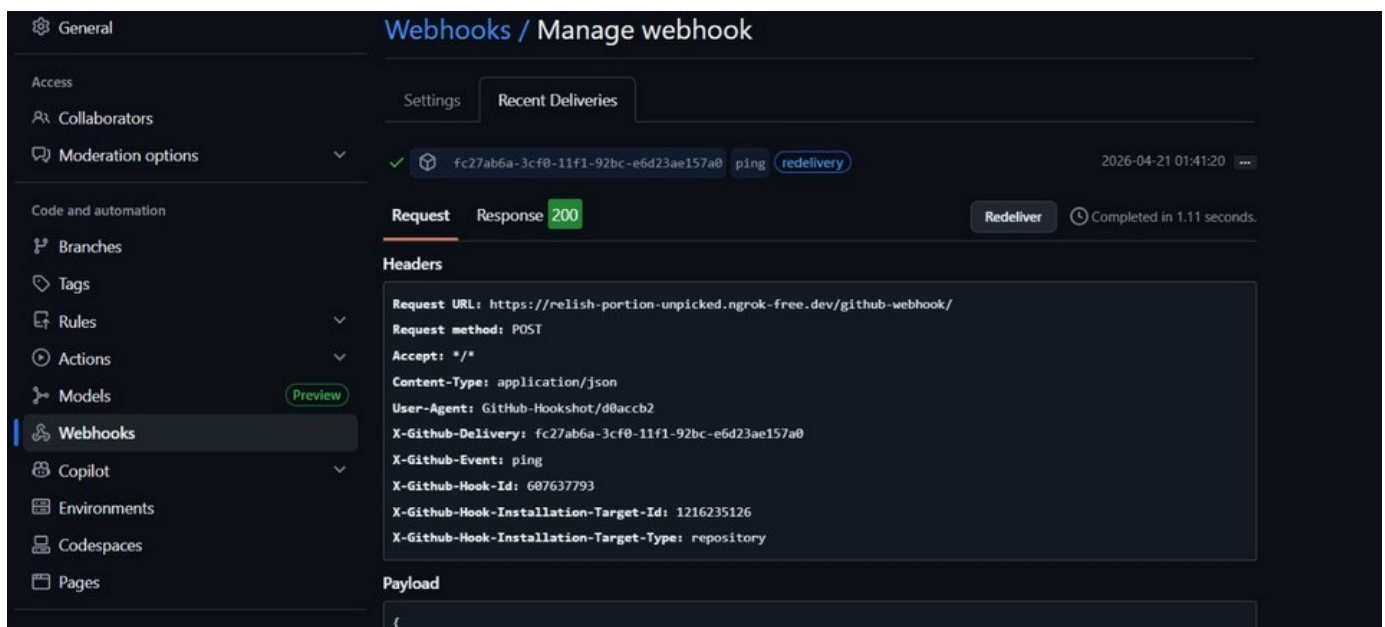
Part D — GitHub Webhook Integration

Configure Webhook in GitHub

Go to your GitHub repository:

Settings → Webhooks → Add Webhook

Field	Value
Payload URL	http://<your-server-ip>:8080/github-webhook/
Content type	application/json
Which events	Just the push event



Enable Webhook Trigger in Jenkins

In your pipeline job → Configure:

- Check “GitHub hook trigger for GITScm polling”

Now every git push to the repository will automatically trigger a new Jenkins build.

Part E — Execution Flow

1. Developer: git push → GitHub
↓
2. GitHub sends HTTP POST → Jenkins (webhook)
↓
3. Jenkins reads Jenkinsfile and executes:
 - | Stage 1 [Clone Source]
| git clone https://github.com/Pavilion15/my-app.git (branch: main)
 - | Stage 2 [Build Docker Image]
| docker build -t anketkiran/myapp:latest .
 - | Stage 3 [Login to Docker Hub]
| echo \$DOCKER_TOKEN | docker login -u anketkiran --password-stdin
 - | Stage 4 [Push to Docker Hub]
| docker push anketkiran/myapp:latest↓
4. Image available on Docker Hub docker pull
anketkiran/myapp:latest ← works from anywhere

Jenkins
/ ci-cd-pipeline
/ #7
/ Stages

< #7
Rerun

Started by Siddharth Negi
Replayed #6
(diff)
Started 1 min 18 sec ago
Queued 2 ms
Took 31 sec

Graph

Checkout SCM 1s
Clone Source 0.96s
Build Docker Image 7s
Login to Docker Hub 2s
Push to Docker Hub 17s

Push to Docker Hub
17s
Started 3m 1s ago
Jenkins

docker push \$IMAGE_NAME:latest
17s

```

0 + docker push siddharthnegi/myapp:latest
1 The push refers to repository [docker.io/siddharthnegi/myapp]
2 bed04db721b4: Waiting
3 5435b2dcdf5c: Waiting
4 ddf3b231f567: Waiting
5 c06c342c1a7c: Waiting

```

Before connected to webhook

Jenkins
/ ci-cd-pipeline
/ #8

Status

</> Changes
Console Output
Edit Build Information
Delete build '#8'
Polling Log
Timings
Git Build Data
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps

#8 (20 Apr 2026, 20:25:56)
Add description
Keep this build forever

Started by GitHub push by Pavilion5
Started 1 min 50 sec ago
Took 27 sec

This run spent:

- 9.2 sec waiting;
- 27 sec build duration;
- 36 sec total from scheduled to completion.

Revision: 99ed7098ee2f857b2507f603b5239d3f3953914b

Repository: https://github.com/siddharth890git/my-app.git

- refs/remotes/origin/main

</>
test webhook
99ed709

siddharthnegi at 20:25 20/04/2026

pipeline

My Hub

Search Docker Hub

Ctrl K

Repositories

All repositories within the **aryanraj** namespace.

Search by repository name

All content

Name	Last pushed	Contains
aryanraj/myapp	3 minutes ago	Image
aryanraj/my-flask-app	2 months ago	Image
aryanraj/testimage	2 months ago	Image

After connected to webhook

Jenkins

ci-cd-pipeline

#8

Stages

Search

Settings

Profile

Start

Checkout SCM

Clone Source

Build Docker Image

Login to Docker Hub

Push to Docker Hub

End

Started by GitHub push by siddharth890git

Started 2 min 0 sec ago

Queued 9.2 sec

Took 27 sec

Changes

Search

Checkout SCM 1s

Clone Source 0.93s

Build Docker Image 9s

Login to Docker Hub 2s

Push to Docker Hub 10s

Push to Docker Hub

10s

Started 1m 43s ago

Jenkins

...

docker push \$IMAGE_NAME:tatest

10s

...

0 + docker push siddharthnegi/myapp:latest

1 The push refers to repository [docker.io/siddharthhegi/myapp]

2 9cefae7dfc6d: Waiting

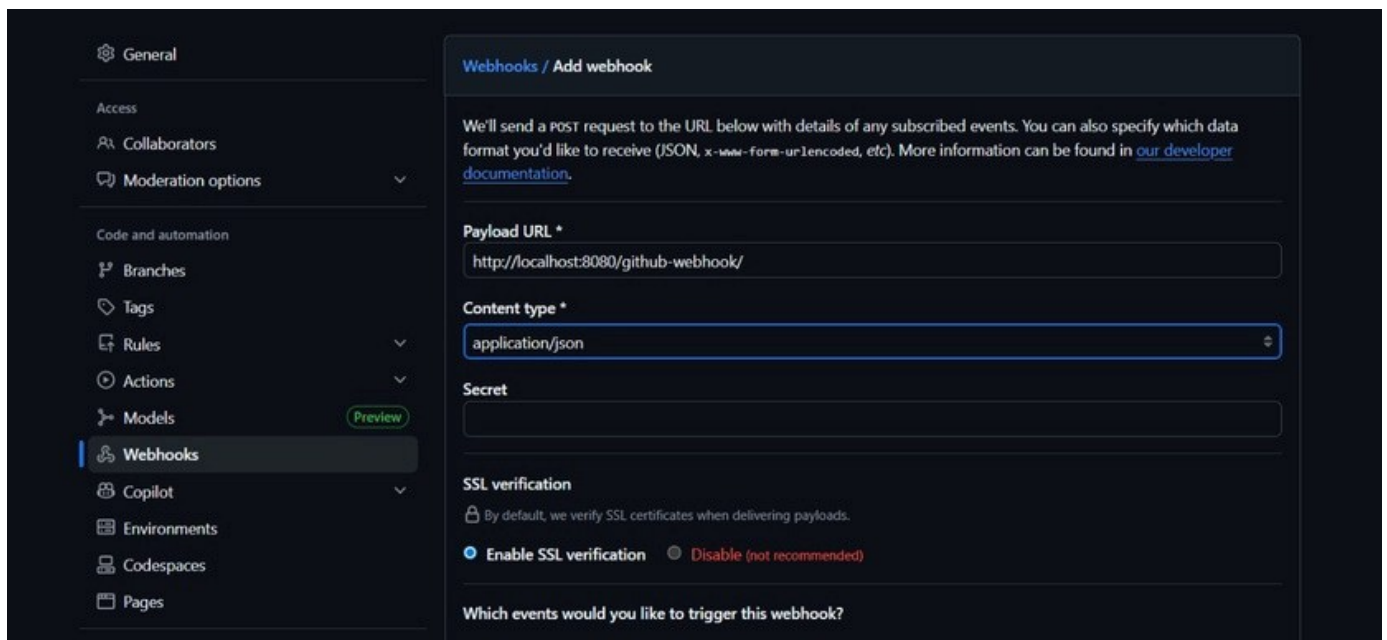
3 97d0a4024ce3: Waiting

4 935d60c8480a: Waiting

5 ddf3b231f567: Waiting

6 4f3bc29c52c2: Waiting

Shared by GitHub Push



Troubleshooting

Issue 1 — localhost / Private IP Not Accessible

When setting up the GitHub webhook, the following URLs were tried:

`http://localhost:8080/github-webhook/` `http://192.168.x.x:8080/github-webhook/`

```
ngrok

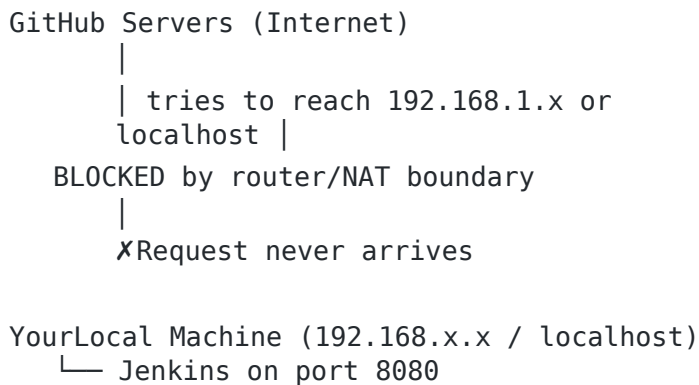
LLMs 4x smaller, 2x faster: https://ngrok.com/blog/quantization

Session Status      online
Account             raghav (Plan: Free)
Version             3.38.0
Region              India (in)
Latency              38ms
Web Interface        http://127.0.0.1:4040
Forwarding            https://relish-portion-unpicked.ngrok-free.dev -> http://localhost:8080

Connections          ttl    opn    rt1    rt5    p50    p90
                     0      0      0.00   0.00   0.00   0.00
```

Why this fails:

192.168.x.x is a private IP address (RFC 1918 range). Private IPs are only routable within your local network (home/office LAN). They are blocked at the network boundary and cannot be reached from the public internet. GitHub's servers sit on the public internet and have no path to your local 192.168.x.x address.



Fixed

Used ngrok to create a public tunnel to the local Jenkins server.

Issue 2 — Webhook Not Publicly Exposed

Jenkins was only running locally. GitHub's webhook delivery failed with:

```
We couldn't deliver this payload
```

This error in GitHub's webhook settings means GitHub sent the HTTP POST request but received no response — the server was unreachable.

Root cause: There was no public URL pointing to the local Jenkins instance. Without a publicly accessible endpoint, GitHub has no way to notify Jenkins about push events. **Fix — Expose Jenkins Using ngrok**

ngrok is a tool that creates a secure tunnel from a public URL on the internet to a port on your local machine. It acts as a relay — GitHub sends the webhook to ngrok's public URL, and ngrok forwards it to your local Jenkins.

Step 1: Run ngrok

```
ngrok http 8080
```

Step 2: Copy the generated public URL ngrok outputs something like:

```
Forwarding https://a1b2c3d4.ngrok-free.app -> http://localhost:8080
```

The `https://a1b2c3d4.ngrok-free.app` URL is now publicly accessible from anywhere on the internet.

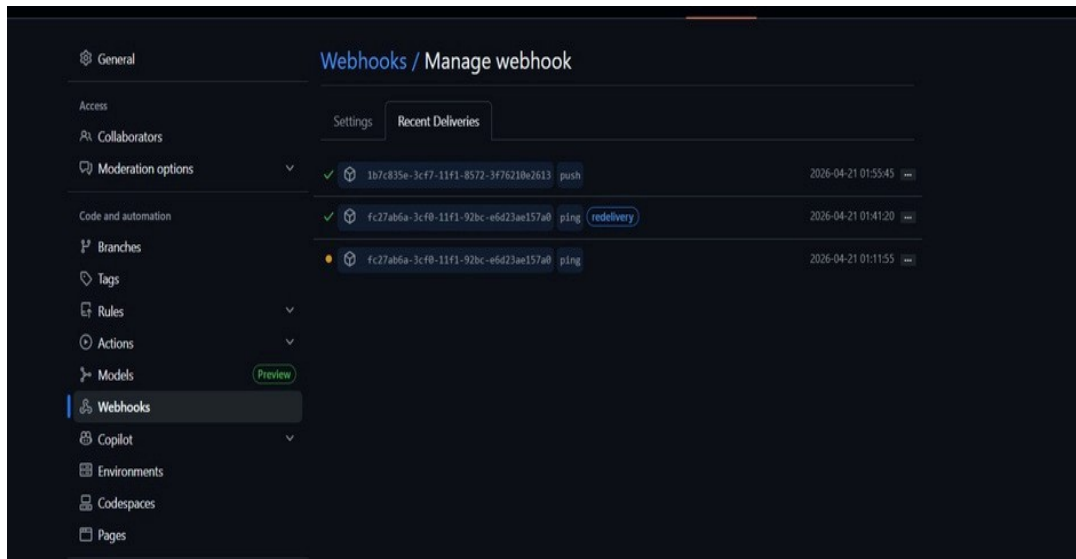
Step 3: Update the GitHub Webhook

URL Go to your GitHub repository:

Settings → Webhooks → Edit existing webhook

Change the Payload URL to:

`https://a1b2c3d4.ngrok-free.app/github-webhook/`



🚩 The `/github-webhook/` path at the end is mandatory — Jenkins listens for GitHub specifically at this

Step 4: Verify ngrok is receiving traffic

In the ngrok terminal or the ngrok web dashboard (`http://127.0.0.1:4040`), check:

```
Connections > 0
HTTP 200 responses
```

A connection count above 0 confirms GitHub's requests are reaching your Jenkins instance.

🟡 Important: The free tier of ngrok generates a new random URL every time you restart it. If ngrok is restarted, the GitHub webhook URL must be updated again with the

```
root@d9f4b7alc3e2:/# apt install docker.io -y
Installing:
  docker.io

Installing dependencies:
  apparmor      libapparmor1      libmnl0      libpython3.13-minimal  python3-minimal
  containerd    libbpf1           libmodule-find-perl  libpython3.13-stdlib  python3-protobuf
  criu          libcap2-bin      libnet1      libreadline8t64       python3-pycru
  dbus         libcomn-2        libnetfilter-contrack3  libsort-naturally-perl  python3.13
  dbus-bin     libcryptsetup12  libnftnlk0    libsystemd-shared     python3.13-minimal
  dbus-daemon  libdbus-1-3      libnftables1  libterm-readkey-perl  readline-common
  dbus-session-bus-common  libdevmapper1.02.1  libnftnl11  libtirpc-common       runc
  dbus-system-bus-common  libelf1t64        libn1-3-200  libtirpc3t64         systemd
  dbus-user-session  libintl-perl      libnss-systemd  libxtables12         systemd-cryptsetup
  dmsetup        libintl-xs-perl  libpam-cap     linux-sysctl-defaults  systemd-sysv
  docker-buildx  libip4tc2        libpam-systemd  media-types          systemd-timesyncd
  docker-cli     libip6tc2        libproc-procstable-perl  needrestart          xz-utils
  gettext-base  libjansson4      libprotobuf-c1  netbase
  iproute2      libjson-c5       libprotobuf32t64  nftables
  iptables      libkmod2         libpython3-stdlib  python3

Suggested packages:
  apparmor-profiles-extra  rinse      libdw1t64      python3-venv      systemd-boot
  apparmor-utils          rootlesskit  libgcrypt20    python3.13-venv  systemd-resolved
  containernetworking-plugins  xfsprogs    libpwquality1  python3.13-doc  systemd-repart
  docker-doc              zfs-fuse    libqrencode4    binutils         libtss2-tcti-device0
  aufs-tools              | zfsutils-linux  needrestart-session  binfmt-support  polkitd
  btrfs-progs             firewallld    | libnotify-bin  readline-doc
  cgroupfs-mount          kmod         uuencode-tool   systemd-container
  debootstrap             libtss2-rc0t64  python3-doc     systemd-homed
  e2fsprogs               libarchive13t64  python3-tk      systemd-userdbd

Summary:
  Upgrading: 0, Installing: 73, Removing: 0, Not Upgrading: 0
  Download size: 100 MB
  Space needed: 401 MB / 216 GB available
```

Issue 3 — Jenkins 403 Forbidden (CSRF Protection)

Even after setting up ngrok and updating the webhook URL, webhook deliveries were still failing.

The GitHub webhook response showed:

```
HTTP 403 Forbidden
```

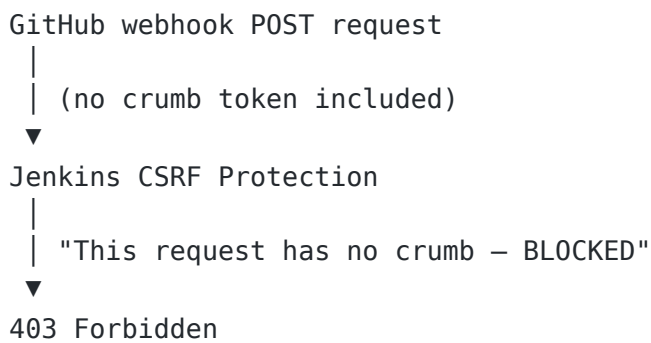
And in Jenkins logs:

```
No valid crumb was included in the request
```

Why this happens:

Jenkins has a built-in security feature called CSRF Protection (Cross-Site Request Forgery Protection). CSRF is a type of web attack where a malicious website tricks a logged-in user's browser into making unauthorized requests to another site (in this case, Jenkins).

To prevent this, Jenkins requires all POST requests to include a special token called a crumb — a one-time security token that proves the request came from a legitimate source (like the Jenkins web UI). GitHub’s webhook does not know about Jenkins’ crumb system, so it sends a plain POST request without one. Jenkins sees a POST request without a crumb and rejects it with 403 Forbidden .



Fix — Disable CSRF Protection in Jenkins

Navigate to:

Manage Jenkins → Security → CSRF Protection

Either:

- Disable the “Prevent Cross Site Request Forgery exploits” option entirely, OR
- Install the “GitHub plugin” which handles CSRF exceptions for GitHub webhooks automatically

To disable CSRF via the UI:

1. Go to Manage Jenkins → Configure Global Security
2. Uncheck “Prevent Cross Site Request Forgery exploits”
3. Click Save

After this change, Jenkins accepts incoming POST requests from GitHub without requiring a crumb token.

 **Security note:** Disabling CSRF protection reduces security. This is acceptable for local learning environments. For production Jenkins, keep CSRF enabled and configure proper GitHub plugin settings instead.

Issue 4 — Wrong Branch Name (master vs main)

Jenkins pipeline failed at the Clone Source stage with an error like:

```
ERROR: Couldn't find any revision to build. Verify the repository and target branch config
```

Or Jenkins was trying to check out:

```
origin/master
```

But the GitHub repository used the default branch

```
name: main
```

Why this happens:

GitHub changed its default branch name from `master` to `main` in October 2020. Older Jenkins configurations, tutorials, and default settings still reference `master`. If the Jenkinsfile or job configuration specifies `master` but the repository only has a `main` branch, Git cannot find the branch and fails.

```
Jenkins requests: git checkout master
GitHub repository: only has branch → main
```

```
Result: Branch not found → Build fails
```

Fix — Update the Branch Name in Jenkinsfile

Change the `git` step in the Jenkinsfile from:

```
git branch: 'master', url: 'https://github.com/Pavilioni5/my-
```

app.git' To:

```
git branch: 'main', url: 'https://github.com/Pavilioni5/my-app.git'
```

Also verify in the Jenkins job configuration:

Go to the pipeline job → Configure → Branch Specifier and set it to:

```
*/main
```

How to check which branch your repository uses:

```
git branch -a
```

Issue 5 — Docker Not Found Inside Jenkins Container

The pipeline failed at the Build Docker Image

```
stage with: docker: not found Or: sh: docker:
```

```
command not found
```

Why this happens:

The official jenkins/jenkins:its Docker image does not include Docker pre-installed. When Jenkins tries to run `docker build` as a shell command, the shell inside the Jenkins container searches for a `docker` binary and cannot find one.

This happens when the `docker.sock` volume was not mounted (or the mount was not working), or when Docker CLI binaries are simply absent inside the container.

```
Jenkins Container
└─ sh 'docker build ...'
    └─ searches for 'docker' binary in PATH
        └─ not found → ERROR
```

Fix — Install Docker Inside the Jenkins Container

Step 1: Enter the running Jenkins container

```
docker exec -it jenkins bash
```

This opens a shell inside the Jenkins container as root.

Step 2: Update the package list and install Docker

```
apt-get update
apt-get install -y docker.io
```

`docker.io` is the Docker package available in Debian/Ubuntu repositories (which the Jenkins image is based on). This installs the Docker CLI and daemon binaries inside the container.

 alt text

Step 3: Verify Docker is now available